

STATISTICAL MACHINE LEARNING (SML2025)

COMPUTER LAB on Backpropagation

Jan Drchal

1 Overview

The goal of this laboratory lab is to implement backpropagation for MultiLayer Perceptron (MLP). Backpropagation is a method computing the gradient $\frac{\partial \mathcal{L}}{\partial \mathbf{w}}$, which is in turn used by optimization methods such as gradient descent in order to learn the network.

In backpropagation we divide the network into layers (modules). Each layer l is specified by the three following messages:

- *forward message* $\mathbf{z}^{l+1} = \mathbf{f}(\mathbf{z}^l)$, describing the function realized by the layer,
- *backward message* $\frac{\partial \mathbf{z}^{l+1}}{\partial \mathbf{z}^l}$, Jacobian matrix giving sensitivities of all layer outputs w.r.t. to all of its inputs and
- *parameter message* $\frac{\partial \mathbf{z}^{l+1}}{\partial \mathbf{w}^l}$, giving sensitivities of all layer outputs w.r.t. to all layer parameters.

Let $\delta^l = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^l}$ be the sensitivity of the loss w.r.t. the module input for layer l , then

$$\delta_i^l = \frac{\partial \mathcal{L}}{\partial z_i^l} = \sum_j \delta_j^{l+1} \frac{\partial z_j^{l+1}}{\partial z_i^l}. \quad (1)$$

Parameter gradient can be then computed using:

$$\frac{\partial \mathcal{L}}{\partial w_i^l} = \sum_j \delta_j^{l+1} \frac{\partial z_j^{l+1}}{\partial w_i^l}. \quad (2)$$

Your task can be summarized by the following steps:

1. Implement all messages for linear, ReLU, and softmax layers as well as for the multinomial cross-entropy loss.
2. Implement the backpropagation.
3. Train MLP classifier using Stochastic Gradient Descent (SGD) on an artificial spiral dataset (see Figure 1).
4. Modify your layer implementations so these can cope with numerical instabilities (e.g., underflow, overflow, or division by zero). This involves defining a compound layer combining softmax with cross-entropy loss. Train MLP classifier on the well known MNIST dataset¹. MNIST is a database of handwritten digits (grayscale images of 28×28 pixels) containing 60,000 training and 10,000 test examples. See Figure 1.

¹<http://yann.lecun.com/exdb/mnist/>

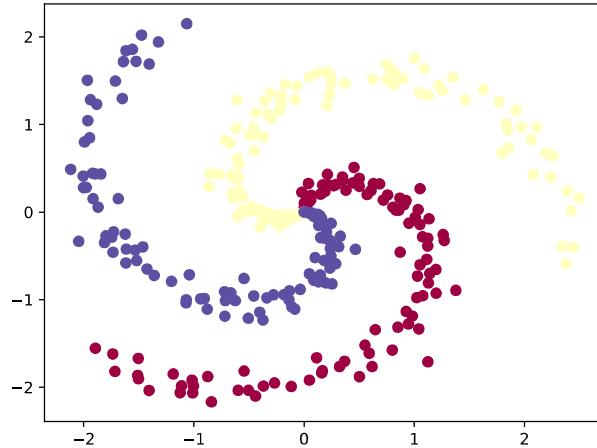


Figure 1: **Spiral dataset.**

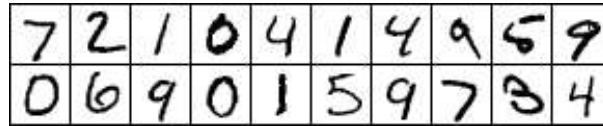


Figure 2: **MNIST digits sample.**

5. Perform experiments.

This homework is worth **10 points**.

2 Task assignment

Assignment 1.

- Implement all messages in `LinearLayer`, `ReLULayer`, `SoftmaxLayer` and `LossCrossEntropy` classes. Note that backward messages are not implemented in separate methods. It is more efficient to compute the δ^l 's (**delta** methods) directly as the Jacobian matrices representing the backward messages are often sparse. Similarly, there is no method dedicated to compute the parameter message, instead, you will find **grad** method computing the gradient of loss w.r.t. all layer parameters. Details are given in the corresponding comments.
- Implement the **gradient** method in `MLP` class performing the actual backpropagation.
- **[Optional]** Train an MLP classifier on the spiral dataset using `experiment_spiral` as a base for your experiments. Use the MLP architecture as well as parameters given in the source file.

Hint: You might want to use a simpler dataset such as XOR for debugging purposes. XOR contains four samples only while it is not linearly separable, which makes it an ideal candidate for early experiments. Use `experiment_XOR` to get started.

Hint: Use matrix operations such as `numpy.dot()` wherever possible to make computations efficient. Use assertions to check correct tensor sizes and/or numerical problems (`numpy.isnan()`, `numpy.isinf()` or `numpy.seterr()` might help).

Assignment 2. [Analyse]

- Find forward and backward messages for a compound layer composed of the softmax layer:

$$p_k(\mathbf{s}) = \frac{e^{s_k}}{\sum_{c=1}^K e^{s_c}} \quad (3)$$

and multinomial cross-entropy loss:

$$\ell = - \sum_{k=1}^K t_k \log(p_k), \quad (4)$$

where $\mathbf{s} = (s_1, \dots, s_K)$, $\mathbf{p} = (p_1, \dots, p_K)$, $k \in \{1, \dots, K\}$ are softmax inputs and outputs, $\mathbf{t} = (t_1, \dots, t_K)$ a one-hot encoded vector of targets and ℓ the value of loss for a single sample.

- Show that softmax is invariant to shift in inputs, i.e., $p_k(\mathbf{s}') = p_k(\mathbf{s})$ where $s'_k = s_k + c$ for $k \in \{1, \dots, K\}$ and $c \in \mathbb{R}$.

Remark: To prevent overflows and underflows in softmax, $\max(\mathbf{s})$ is often subtracted from all elements of its input $\mathbf{s}$. Subtracting $\max(\mathbf{s})$ makes all elements of $\mathbf{s}$ non-positive which prevents overflow. At least one element becomes zero which makes denominator greater or equal to one.

Assignment 3.

- Reimplement softmax according to the previous remark.
- Implement the compound layer (`LossCrossEntropyForSoftmaxLogits` class) from Assignment 2 and use it instead of separate softmax and loss layers.
- **[Optional]** Train a deeper architecture as defined in `experiments_MNIST` for MNIST dataset.
- **[Optional]** Print a plot showing the development of mean weight amplitude (mean absolute value) for each linear layer over training epochs. Show values normalized w.r.t. the initial mean amplitude.

Remark: Note that you should use the compound layer for training only. In the network evaluation phase (`MLP.propagate()`) you should use standard softmax layer to get actual predictions. These two regimes are facilitated by `output_layers` parameter in `MLP` constructor as well as by the `MLP.propagate()` method's parameter of the same name.

Remark: The size of the network and other parameters in `experiment_MNIST` are chosen in order to cause numerical instability² for the naïve implementation from Assignment 1. You should be able to get reasonable and even higher accuracy using much fewer neurons in the hidden layer still.

²Note that the numerical instability might not demonstrate itself in some cases due to specific implementation details in your code.